Line Following Robot

Maitrey Amrutiya, Javier Briceno, Eduardo Harari, Keayen Jeppson, Kevin Patel, and Nicholas Reichardt
(All MXET Group)

Problem Statement

In many industrial, and service settings, there is a growing need for small autonomous robots that can reliably navigate along predefined paths without continuous human input. However, low-cost solutions often struggle with consistent line detection, tracking through curves, and maintaining stability.

This project addresses the need for a simple, affordable, and reliable line-following robot that can be used for teaching controller concepts and prototyping automation tasks on a scalable level. This also demonstrates clear and measurable performance in tracking a visual path on the floor.

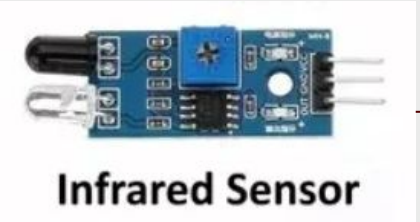
Existing Solution

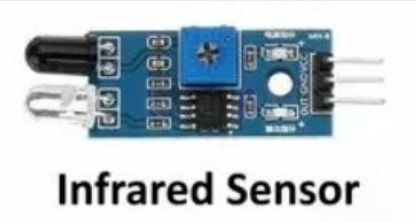
Many low-cost-line-following robot kits use simple on/off logic with fixed motor speeds. Sensors only detect when the robot is on or off the line, and the controller reacts with basic left/right motor commands, without a designed closed-loop system. These systems can follow the line in very ideal conditions and produce jerky motion and overshoot curves.

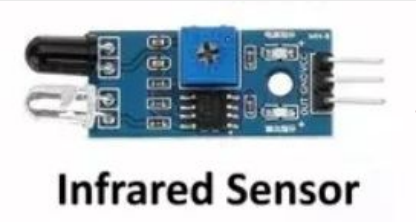
Proposed Solution

We propose a closed-loop line-following robot that uses three front IR sensors to lateral line position and two rear DC motors for differential drive, controlled by an Arduino. Sensor Voltages are combined into an single error signal, and a discrete time controller (PD) adjusts the motor speeds in real time to minimize tracking error. This approach provides smoother motion, and better reliability on curves.

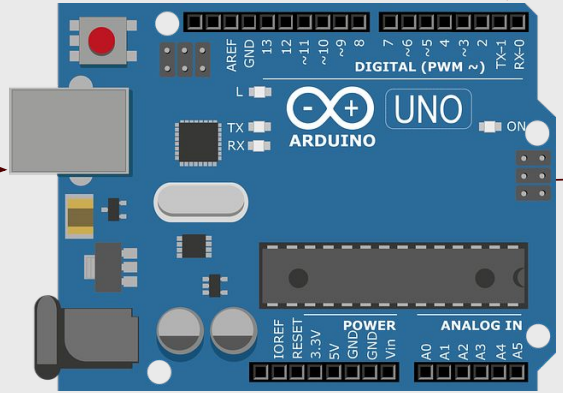
Diagram



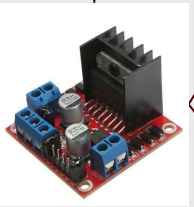




Battery



Arduino Microcontroller

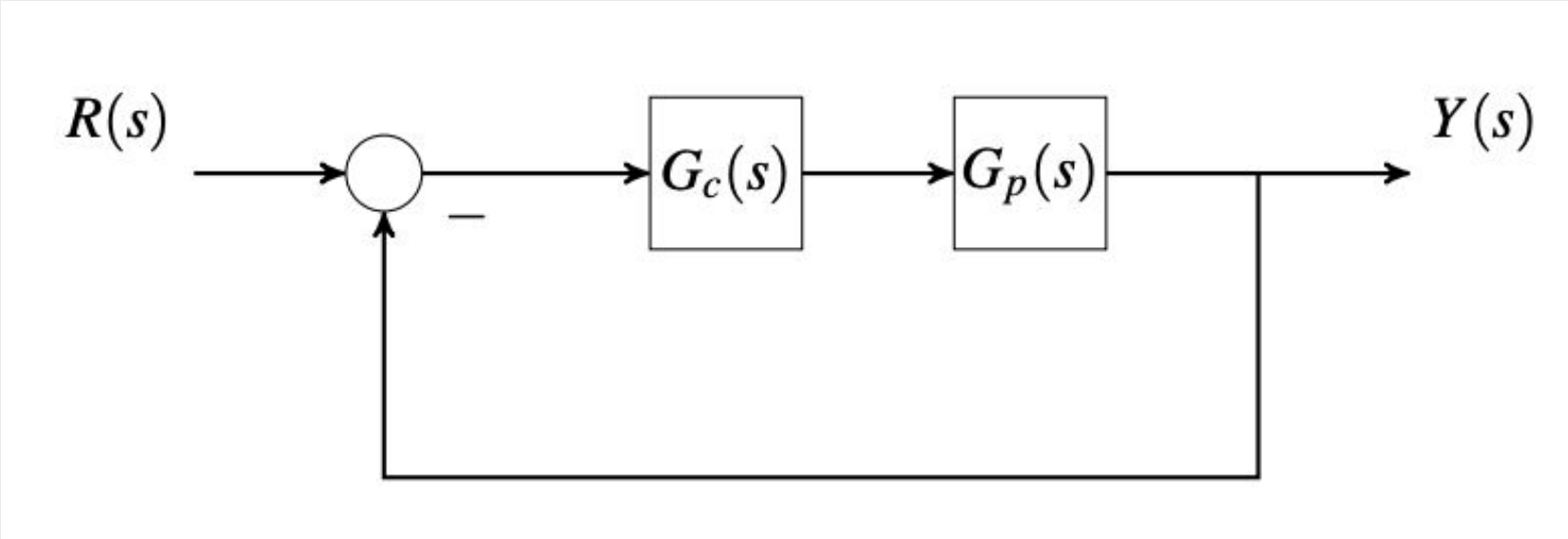


H-Bridge

Right Motor

Left Motor

System Block Diagram



Transfer Function

- Modeled the line following robot as a system from steering command *u(t)* to line position error *y(t).*
- The plant dynamics are approximated by a first-order lag plus an integrator.

$$G(s) = \frac{k}{(\tau s+1)s}$$

- For our robot
  - k=1.72, τ=0.16
- With a PD controller Chosen ζ and
  - C(s)=Kp+Kds
- The closed-loop transfer function becomes

$$\tau(s) = \frac{c(s)G(s)}{1 + c(s)g(s)}$$

Chosen ζ and ω_n

$\zeta = 1.12$

$\omega_n = 11.35$

Transfer Function

- Substituting *C(s) and G(s)*

$$\frac{(k_p+k_d(s))(\frac{k}{(\tau s+1)s})}{1+(k_p+k_d(s))(\frac{k}{(\tau s+1)s})} \longrightarrow \frac{(k_p+k_d(s))(\frac{k}{\tau})}{s^2+(\frac{k_d k+1}{\tau})s+\frac{k_p k}{\tau}}$$

- This gives a second-order form, which we match to

$$s^2 + 2\zeta_n \omega_n s + \omega_n^2$$

- Allows us to find Kp and Kd with approximations to optimize

$$k_p = \frac{\omega_n^2}{10.75}$$

$$k_d = \frac{2\tau\omega_n\zeta_n - 1}{k}$$

Controller Analysis

- Approximating zeta and Wn values to optimize robot operation.

Zeta	Wn (rad/sec)	Kp	Kd
0.4	10.1	9.484394703	0.1701445182
0.6	10.4	10.05619185	0.5792351632
0.8	10.7	10.64472453	1.010639843
1	11	11.24999274	1.464358559
1.1	11.3	11.87199647	1.730267478
1.12	11.35	11.97729082	1.782705461
1.2	11.4	12.08305005	1.962705344

← Optimal

Control Design

1. Control Strategy
   a. Three discrete IR inputs are converted into a single error signal
   b. Implemented a PD controller to balance tracking speed (P) with stability (D)
   c. Closed loop control aims to minimize the deviation between the robot's center and the line
2. Design Methodology
   a. Combined the plant dynamics G(s) with the controller C(s) for the closed loop transfer function
   b. Equated the denominator of the closed-loop system to the standard second-order characteristic equation: $s^2 + 2\zeta\omega_n s + \omega_n^2$
   c. This derivation allowed us to solve for K_p and K_d algebraically

Budget

Item	Price $
Arduino UNO R3	$17
DIY Robot Chassis Kit with Motors and Wheels	$21
L298N Motor Driver	$3
IR Sensors	$4
Tape	$4
3D Printed Parts	$2
Total Price	**$51**

Total Effort

Task	Hours
Build	2
Calculations and Function	1
Software	2
Testing & PD Control Tuning	1.5
Assignments & MISC	1
TOTAL	**7.5**

Implementation

The PD controller design was used to control the speed of the left and right motors based on the error that was given by the three IR sensors in the front of the robot.

Program was flashed and run on the Arduino Uno R3 using a 9V Battery to power the board, while using 4 AA Batteries to power the Motor Driver.

Results

The robot is able to turn and follow a line that has a sharp turn of no more than 20 degrees.

The robot can also follow an arc of a radius of no less than 10 inches.

Error is due to low quantity and sensitivity of the sensors.

What We Learned

- How to design a closed loop control system
- How to implement physical components into an equation
- How to troubleshoot to optimize a system